

Controllo del flusso

break e continue

Come interrompere o saltare un passaggio dentro un ciclo.

Controllare meglio un ciclo

A volte un ciclo non deve arrivare alla sua fine naturale.

Java offre due parole utili:

- `break` interrompe il ciclo
- `continue` salta al giro successivo

Usale con misura: rendono alcuni casi più semplici, ma se abusate possono rendere il flusso difficile da seguire.

break

`break` esce subito dal ciclo.

```
for (int i = 1; i <= 10; i++) {  
    if (i == 4) {  
        break;  
    }  
  
    System.out.println(i);  
}
```

Output:

```
1  
2  
3
```

Quando `i` vale `4`, Java esegue `break` e il ciclo finisce.

Cercare un valore

`break` è utile quando hai trovato quello che cercavi.

```
int[] numeri = {4, 8, 15, 16, 23, 42};
int cercato = 16;
boolean trovato = false;

for (int i = 0; i < numeri.length; i++) {
    if (numeri[i] == cercato) {
        trovato = true;
        break;
    }
}

System.out.println(trovato);
```

Output:

```
true
```

Dopo aver trovato `16`, non serve controllare gli altri numeri.

continue

`continue` salta il resto del blocco e passa al prossimo giro.

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) {
        continue;
    }

    System.out.println(i);
}
```

Output:

```
1
2
4
5
```

Quando `i` vale `3`, Java salta la stampa e continua con `4`.

Saltare valori non utili

Esempio: stampiamo solo i numeri positivi.

```
int[] numeri = {5, -2, 8, -1, 0, 3};

for (int i = 0; i < numeri.length; i++) {
    if (numeri[i] <= 0) {
        continue;
    }

    System.out.println(numeri[i]);
}
```

Output:

```
5
8
3
```

I numeri minori o uguali a zero vengono saltati.

break in un while

Puoi usare `break` anche con `while`.

```
public class Menu {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        while (true) {  
            System.out.print("Scrivi esci per terminare: ");  
            String comando = scanner.nextLine();  
  
            if (comando.equals("esci")) {  
                break;  
            }  
  
            System.out.println("Hai scritto: " + comando);  
        }  
  
        scanner.close();  
    }  
}
```

`while (true)` crea un ciclo infinito intenzionale. `break` decide quando fermarlo.

Regola pratica

Usa `break` quando:

- hai trovato un risultato
- l'utente vuole uscire
- continuare sarebbe inutile

Usa `continue` quando:

- vuoi saltare solo un caso
- il resto del ciclo non serve per quel giro

Se il codice diventa difficile da leggere, prova prima a riscrivere la condizione del ciclo.

Condizioni

Come prendere decisioni nel codice con `if`, `else` e `switch`.

A cosa servono le condizioni

Una condizione permette al programma di scegliere cosa fare.

E come un bivio:

- se succede una cosa, fai questo
- altrimenti, fai qualcos'altro

In Java le condizioni usano valori `boolean` : `true` o `false` .

if

`if` esegue un blocco solo se la condizione è vera.

```
int eta = 20;

if (eta >= 18) {
    System.out.println("Puoi entrare.");
}
```

Output:

```
Puoi entrare.
```

La condizione è:

```
eta >= 18
```

Se vale `true` , Java esegue il blocco tra parentesi graffe.

else

`else` indica cosa fare quando la condizione è falsa.

```
int eta = 16;

if (eta >= 18) {
    System.out.println("Puoi entrare.");
} else {
    System.out.println("Non puoi entrare.");
}
```

Output:

```
Non puoi entrare.
```

else if

Quando hai piu casi, usa `else if` .

```
int voto = 7;

if (voto >= 9) {
    System.out.println("Ottimo");
} else if (voto >= 6) {
    System.out.println("Sufficiente");
} else {
    System.out.println("Da riprovare");
}
```

Output:

```
Sufficiente
```

Java controlla dall'alto verso il basso. Appena trova una condizione vera, esegue quel blocco e salta gli altri.

Condizioni con operatori logici

Puoi combinare piu condizioni.

```
int eta = 20;
boolean haBiglietto = true;

if (eta >= 18 && haBiglietto) {
    System.out.println("Ingresso consentito.");
}
```

`&&` significa "e": entrambe le condizioni devono essere vere.

Con `||`, invece, basta una condizione vera:

```
boolean haPassword = false;
boolean haCodiceTemporaneo = true;

if (haPassword || haCodiceTemporaneo) {
    System.out.println("Accesso possibile.");
}
```

switch

`switch` e comodo quando confronti una stessa variabile con piu valori precisi.

```
int giorno = 2;

switch (giorno) {
    case 1:
        System.out.println("Lunedì");
        break;
    case 2:
        System.out.println("Martedì");
        break;
    case 3:
        System.out.println("Mercoledì");
        break;
    default:
        System.out.println("Giorno non valido");
}
```

Output:

Martedì

break ferma lo **switch** dopo il caso trovato.

default viene eseguito se nessun caso corrisponde.

Quando usare if e quando switch

Usa **if** quando hai condizioni diverse:

```
if (eta >= 18 && haBiglietto) {
    ...
}
```

Usa **switch** quando controlli molti valori dello stesso dato:


```
switch (scelta) {  
    case 1:  
        ...  
}
```

Esempio completo

```
public class Menu {  
    public static void main(String[] args) {  
        Scanner scanner = new Scanner(System.in);  
  
        System.out.println("1. Saluta");  
        System.out.println("2. Mostra aiuto");  
        System.out.print("Scelta: ");  
  
        int scelta = scanner.nextInt();  
  
        switch (scelta) {  
            case 1:  
                System.out.println("Ciao!");  
                break;  
            case 2:  
                System.out.println("Scegli un numero dal menu.");  
                break;  
            default:  
                System.out.println("Scelta non valida.");  
        }  
  
        scanner.close();  
    }  
}
```

Le condizioni sono uno dei primi strumenti che trasformano un programma da sequenza fissa a programma capace di reagire.

Ciclo for-each

Come attraversare array e collezioni senza gestire manualmente gli indici.

Un modo piu semplice per attraversare dati

Quando vuoi leggere tutti gli elementi di un array o di una collezione, spesso non ti serve l'indice.

In questi casi puoi usare il ciclo for-each.

```
String[] nomi = {"Luca", "Sara", "Mina"};

for (String nome : nomi) {
    System.out.println(nome);
}
```

Output:

```
Luca
Sara
Mina
```

Leggere la sintassi

```
for (String nome : nomi) {
    ...
}
```

Si legge così:

"per ogni `String` chiamata `nome` dentro `nomi`, esegui questo blocco".

`nome` è una variabile temporanea. A ogni giro contiene un elemento diverso.

for-each con array

```
int[] voti = {7, 8, 6, 9};

for (int voto : voti) {
    System.out.println(voto);
}
```

Output:

```
7
8
6
9
```

Il codice è più breve rispetto a un `for` con indice.

for-each con ArrayList

```
public class Lista {
    public static void main(String[] args) {
        ArrayList prodotti = new ArrayList<>();
        prodotti.add("Pane");
        prodotti.add("Latte");
        prodotti.add("Mele");

        for (String prodotto : prodotti) {
            System.out.println(prodotto);
        }
    }
}
```

Output:

Pane
Latte
Mele

Quando e comodo

Il for-each e comodo quando vuoi:

- stampare tutti gli elementi
- sommare tutti i numeri
- cercare un valore
- controllare ogni elemento senza usare la posizione

Esempio:

```
int[] prezzi = {10, 20, 15};  
int totale = 0;  
  
for (int prezzo : prezzi) {  
    totale = totale + prezzo;  
}  
  
System.out.println(totale);
```

Output:

45

Limite: non hai l'indice

Con for-each non hai direttamente la posizione dell'elemento.

Se devi stampare anche l'indice, usa un `for` normale:

```
String[] nomi = {"Luca", "Sara", "Mina"};

for (int i = 0; i < nomi.length; i++) {
    System.out.println(i + ": " + nomi[i]);
}
```

Output:

```
0: Luca
1: Sara
2: Mina
```

Limite: modificare elementi

Con un array di numeri, questa modifica non cambia l'array originale:

```
int[] numeri = {1, 2, 3};

for (int numero : numeri) {
    numero = numero * 2;
}

for (int numero : numeri) {
    System.out.println(numero);
}
```

Output:

```
1
2
3
```

numero è una copia temporanea del valore.

Se vuoi modificare l'array, usa l'indice:

```
for (int i = 0; i < numeri.length; i++) {  
    numeri[i] = numeri[i] * 2;  
}
```

Regola pratica

Usa for-each quando devi leggere tutti gli elementi.

Usa `for` con indice quando devi conoscere o modificare la posizione.

Ciclo for

Come ripetere istruzioni quando sai quante volte eseguirle.

Ripetere un'azione

Un ciclo serve a ripetere lo stesso blocco di codice.

Il ciclo `for` è utile quando sai quante volte vuoi ripetere.

```
for (int i = 1; i <= 5; i++) {  
    System.out.println(i);  
}
```

Output:

```
1  
2  
3  
4  
5
```

Le tre parti del for

Il ciclo ha questa forma:

```
for (inizializzazione; condizione; aggiornamento) {  
    // codice da ripetere  
}
```

Nell'esempio:

```
for (int i = 1; i <= 5; i++) {
```

succedono tre cose:

- `int i = 1` crea il contatore
- `i <= 5` dice fin quando continuare
- `i++` aumenta `i` di 1 dopo ogni giro

Contare da zero

In programmazione si conta spesso da zero, soprattutto con array e liste.

```
for (int i = 0; i < 3; i++) {  
    System.out.println("Giro " + i);  
}
```

Output:

```
Giro 0  
Giro 1  
Giro 2
```

La condizione `i < 3` produce tre giri: 0, 1, 2.

Usare il for per sommare

```
int totale = 0;

for (int i = 1; i <= 5; i++) {
    totale = totale + i;
}

System.out.println(totale);
```

Output:

15

Il programma somma:

1 + 2 + 3 + 4 + 5

Attraversare un array

Un uso molto comune del `for` è leggere gli elementi di un array.

```
String[] nomi = {"Luca", "Sara", "Mina"};

for (int i = 0; i < nomi.length; i++) {
    System.out.println(nomi[i]);
}
```

Output:


```
Luca  
Sara  
Mina
```

`nomi.length` contiene il numero di elementi dell'array.

Gli indici partono da `0`, quindi l'ultimo indice è `length - 1`.

Errore comune: andare oltre l'ultimo elemento

Questo codice è sbagliato:

```
String[] nomi = {"Luca", "Sara", "Mina"};  
  
for (int i = 0; i <= nomi.length; i++) {  
    System.out.println(nomi[i]);  
}
```

La condizione `i <= nomi.length` permette a `i` di arrivare a `3`, ma gli indici validi sono `0`, `1`, `2`.

La forma corretta è:

```
for (int i = 0; i < nomi.length; i++) {  
    System.out.println(nomi[i]);  
}
```

Ciclo all'indietro

Puoi anche contare al contrario:

```
for (int i = 5; i >= 1; i--) {  
    System.out.println(i);  
}
```

Output:

```
5
4
3
2
1
```

Quando usare for

Usa `for` quando:

- conosci il numero di ripetizioni
- hai un contatore
- vuoi attraversare un array con gli indici

Se invece vuoi ripetere finché una condizione resta vera, spesso è più naturale usare `while`.

Ciclo while

Come ripetere istruzioni finché una condizione resta vera.

Ripetere finché una condizione è vera

Il ciclo `while` ripete un blocco finché una condizione resta vera.

```
int numero = 1;

while (numero <= 3) {
    System.out.println(numero);
    numero++;
}
```

Output:

```
1  
2  
3
```

Java controlla la condizione prima di ogni giro.

Le parti di un while

Un ciclo `while` ha questa forma:

```
while (condizione) {  
    // codice da ripetere  
}
```

Nel nostro esempio:

```
while (numero <= 3)
```

il ciclo continua finché `numero` è minore o uguale a 3.

Dentro il ciclo aggiorniamo la variabile:

```
numero++;
```

Senza questo aggiornamento, `numero` resterebbe sempre uguale.

Il rischio del ciclo infinito

Questo codice non finisce mai:

```
int numero = 1;

while (numero <= 3) {
    System.out.println(numero);
}
```

`numero` resta sempre `1` , quindi la condizione e sempre vera.

Un ciclo che non finisce si chiama **ciclo infinito**.

Attenzione: quando scrivi un `while` , chiediti sempre: "dentro il ciclo, cosa cambia la condizione?"

Usare while con input utente

`while` e molto utile quando non sai prima quante volte l'utente fara qualcosa.

```
public class Password {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String password = "";

        while (!password.equals("java")) {
            System.out.print("Password: ");
            password = scanner.nextLine();
        }

        System.out.println("Accesso consentito.");
        scanner.close();
    }
}
```

Il ciclo continua finche la password non e `"java"` .

`!password.equals("java")` significa: "la password non e uguale a java".

Contare tentativi

Possiamo aggiungere un contatore:

```
public class Tentativi {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        String password = "";
        int tentativi = 0;

        while (!password.equals("java")) {
            System.out.print("Password: ");
            password = scanner.nextLine();
            tentativi++;
        }

        System.out.println("Tentativi usati: " + tentativi);
        scanner.close();
    }
}
```

Ogni giro aumenta `tentativi` di 1.

while o for?

Usa `for` quando sai già quante volte ripetere:

```
for (int i = 0; i < 5; i++) {
    ...
}
```

Usa `while` quando vuoi ripetere finché succede qualcosa:

```
while (!password.equals("java")) {
    ...
}
```

Un ciclo che potrebbe non partire

Poiche `while` controlla la condizione all'inizio, il blocco potrebbe non essere eseguito nemmeno una volta.

```
int numero = 10;

while (numero < 5) {
    System.out.println(numero);
}
```

Qui non viene stampato nulla, perche la condizione e falsa gia all'inizio.